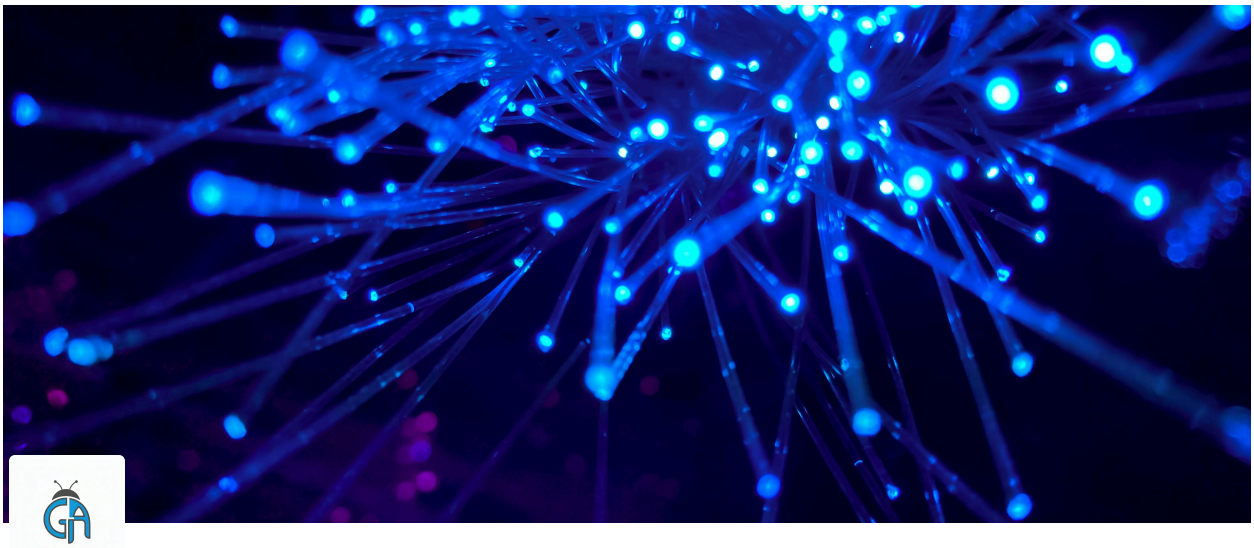

🤝 ATDD

Building the Right Thing First

✍️ **Author:** Georgia Antoniou

🏷️ **Tags:** QA, Agile, Methodology

⏱️ **Reading Time:** 3 min

🚀 TL;DR

Traditional Development: Write Code ➡️ Write Tests ➡️ Fix Bugs.

ATDD: Write Tests ➡️ Write Code ➡️ Pass Tests.

It forces the whole team to agree on "What is the goal?" *before* a single line of code is written.

🧐 The Core Concept

ATDD is a collaborative practice. It brings the **"Three Amigos"** together before the sprint starts:

1. **The Product Owner** (Business)
2. **The Developer** (Tech)
3. **The QA Tester** (Quality)

Instead of writing a vague requirement document, they write the **Acceptance Tests** together. These tests become the requirement. The Developer's goal is simply: "Make this test pass."

🧵 The Tailor Analogy

Traditional Approach:

You tell the tailor: "Make me a nice blue suit."

He sews it. You try it on. "No, I wanted a *dark* blue suit with 3 buttons!"

He has to rip it apart and start over. (Waste).

ATDD Approach:

You, the tailor, and the fabric supplier sit down first.

You draw the suit. You pick the exact fabric swatch. You agree on the button count.

Then he sews it.

The suit fits perfectly the first time.

🔄 The Workflow (The 4 D's)

1. 🗣️ **Discuss:** The Three Amigos talk about the user story. QA asks edge-case questions.
2. 🧪 **Distill:** The team writes the specific Acceptance Tests (often in Gherkin: Given/When/Then).
3. 🔨 **Develop:** The developer writes code *only* to make those tests pass.
4. 🎉 **Demo:** The team runs the tests. If they pass, the story is done.

💡 Real World Example: The "Free Shipping" Logic

The Meeting:

- **PO:** "I want free shipping for expensive orders."
- **QA:** "What is 'expensive'? $50 or $100?"
- **Dev:** "Does that include tax or just the subtotal?"

Because they are talking BEFORE coding, they clarify these rules immediately.

The Agreed Test (The Requirement):

- **Scenario:** Order is exactly $50 before tax.
- **Given** user has $50 of items in cart
- **And** tax is $5
- **When** they checkout
- **Then** shipping cost should be $0.

Now the Developer knows exactly what to build. No guessing.

🧠 Why This Matters

ATDD shifts testing **Left** (earlier in the process).

It prevents the most expensive type of bug: **Misunderstanding the Requirement.**

We don't just "Check the quality" at the end; we "Build quality in" from the start.